

Complete Backend & Database Integration Guide

Step-by-step manual for connecting this project to Supabase & deploying on Vercel

1. Architectural Overview & Stack

This e-commerce application is built on modern Next.js 15 (App Router) paired with a Supabase PostgreSQL backend. It delivers a full re-commerce experience: authentication (email/password & Google OAuth), persistent shopping carts, certified device catalog, doorstep valuation scheduling, and express multi-method checkout with order tracking.

- Frontend Stack: Next.js 15 with React 19, TypeScript, and Tailwind CSS
- Database & Auth: Supabase (PostgreSQL with Row Level Security, Auth & JSONB support)
- Backend API: Next.js App Router API Routes (/api/products, /api/orders, /api/valuations)
- Hosting & Deployment: Vercel Serverless Platform with automated CI/CD and edge proxy

2. Step 1: Create Your Supabase Project

Your friend needs to set up their own database and auth engine on Supabase. Follow these exact steps:

- 2.1 Account: Go to <https://supabase.com> and sign up or log in.
- 2.2 Project Creation: Click 'New Project'. Choose an organization, enter a Project Name (e.g., 'retech-store'), and create a strong Database Password (store this securely).
- 2.3 Region: Select the cloud region closest to your primary audience (e.g., Singapore 'ap-southeast-1' or Mumbai 'ap-south-1' for low latency).
- 2.4 Provisioning: Wait approximately 1-2 minutes for Supabase to provision the PostgreSQL instance.

3. Step 2: Retrieve API Credentials

Once the project is ready, navigate to the Project Settings to copy the required connection variables:

- 3.1 Settings Location: In the Supabase dashboard sidebar, click 'Project Settings' (the gear icon) -> 'API'.
- 3.2 Project URL: Copy the 'Project URL' (format: <https://<project-ref>.supabase.co>).
- 3.3 Public Key: Copy the 'anon' public key or the modern 'publishable' key (starts with sb_publishable_... or eyJ...).

```
# Required Environment Variables Template
NEXT_PUBLIC_SUPABASE_URL=https://<your-project-id>.supabase.co
NEXT_PUBLIC_SUPABASE_ANON_KEY=<your-anon-or-publishable-key>
NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY=<your-anon-or-publishable-key>
```

4. Step 3: Run Database Schema & Migrations

The codebase includes a complete production database schema file located at /supabase/schema.sql. This creates all tables, constraints, foreign keys, RLS security policies, and initial product catalog seed data.

- 4.1 Open SQL Editor: In your Supabase project dashboard, click on 'SQL Editor' in the left menu.
- 4.2 New Query: Click '+ New query' to open a fresh script editor.

- 4.3 Paste Script: Copy the entire contents of `/supabase/schema.sql` and paste it into the editor.
- 4.4 Execute: Click the 'Run' button (or press Ctrl + Enter). All 8 tables and seeds will be instantiated within seconds.

Tables created by the schema:

- - profiles: public.profiles: Stores user contact info, phone, and delivery address.
- - categories: public.categories: Smartphones, Laptops, Audio, Tablets, Wearables, Gaming.
- - products: public.products: Hardware specifications, price, tested points, warranty, and stock.
- - orders: public.orders: Full checkout records, order numbers, shipping address JSON, payment status.
- - cart_items: public.cart_items: Persistent multi-device shopping cart for authenticated users.
- - wishlist_items: public.wishlist_items: Saved products wishlist per user account.
- - sell_requests: public.sell_requests: Doorstep diagnostic device valuation requests and payout records.
- - reviews: public.reviews: Verified customer ratings and feedback.

5. Step 4: Configure Supabase Authentication Providers

To allow users to register, sign in, and maintain session tokens:

- 5.1 Email Auth: Navigate to Authentication -> Providers -> Email. Ensure 'Enable Email provider' is toggled ON.
- 5.2 Email Confirmations: (Recommended for prototyping) Toggle OFF 'Confirm email' so users can sign in immediately without waiting for verification emails.
- 5.3 Google OAuth: (Optional) To enable Google Sign-In, turn on Google Provider, enter your Google OAuth Client ID and Secret from Google Cloud Console, and register the callback URL shown by Supabase.
- 5.4 Redirect URLs: Navigate to Authentication -> URL Configuration. Set 'Site URL' to your Vercel URL (e.g., `https://your-project.vercel.app`) and add '`https://your-project.vercel.app/**`' and '`http://localhost:3000/**`' to Redirect URLs.

6. Step 5: Local Testing with .env.local

Before pushing to Vercel, test the application locally on your machine:

- 6.1 Create .env.local: In the project root, create a file named `.env.local` (this is git-ignored for safety).
- 6.2 Add Keys: Add the Supabase URL and Publishable/Anon key copied in Step 2.
- 6.3 Install: Install all dependencies: `npm install`
- 6.4 Launch: Run the development server: `npm run dev`
- 6.5 Verify: Open `http://localhost:3000` in your browser. Click 'Login' on the top navbar, create an account, place a test order, and verify the order appears in Supabase 'orders' table.

7. Step 6: Deploying to Vercel (Production)

Deploying this Next.js e-commerce app to Vercel is seamless with Git integration:

- 7.1 Push to Git: Commit your code to Git and push it to a GitHub, GitLab, or Bitbucket repository.
- 7.2 Import Project: Log in to `https://vercel.com`, click 'Add New...' -> 'Project', and select your Git repository.
- 7.3 Preset: Under 'Framework Preset', ensure Next.js is selected (Vercel detects this automatically).
- 7.4 Environment Variables: Expand the 'Environment Variables' section and add the exact same three keys:

Key: NEXT_PUBLIC_SUPABASE_URL	Value: https://<your-project>.supabase.co
Key: NEXT_PUBLIC_SUPABASE_ANON_KEY	Value: <your-anon-or-publishable-key>
Key: NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY	Value: <your-anon-or-publishable-key>

- 7.5 Build & Deploy: Click 'Deploy'. Vercel builds the Next.js pages and API routes in ~60 seconds.
- 7.6 Copy Domain: Copy your generated production URL (e.g., https://retech-store.vercel.app).
- 7.7 Update Supabase: Return to your Supabase Dashboard -> Authentication -> URL Configuration -> Redirect URLs, and add your Vercel URL so authentication redirects work seamlessly.

8. Architecture Details & Key Files Reference

Here is how the codebase connects the frontend to the database:

- - Browser Client: lib/supabase/client.ts: Initializes browser-side Supabase client via @supabase/ssr.
- - Server Client: lib/supabase/server.ts: Initializes server-side authenticated client using Next.js cookies().
- - Proxy / Session: lib/supabase/proxy.ts: Manages session refresh while exempting /api routes from redirection.
- - Orders Service: lib/services/ordersService.ts: Handles order creation in Supabase with automatic local fallback.
- - Valuations Service: lib/services/valuationsService.ts: Manages doorstep device sell booking submissions.
- - Products API: app/api/products/route.ts: Serves filtered, sorted products from Supabase catalog.
- - Orders API: app/api/orders/route.ts: Securely processes customer checkout submissions and queries user orders.
- - Schema Script: supabase/schema.sql: Contains full DDL, RLS policies, and product catalog seed rows.

9. Troubleshooting & FAQ

- Issue: Products or Auth not working on Vercel: Check that NEXT_PUBLIC_SUPABASE_URL and NEXT_PUBLIC_SUPABASE_ANON_KEY are added to Vercel Environment Variables, and trigger a Redeploy under Deployments.
- Issue: Can't sign in after signup: In Supabase -> Authentication -> Providers -> Email, disable 'Confirm email' for instant login.
- Issue: Permission denied error from Supabase: Ensure public RLS policies are enabled as defined in schema.sql, or inspect the browser console network tab.
- Issue: Google login redirects to localhost: Ensure the Vercel domain has been added to Supabase Authentication -> URL Configuration -> Redirect URLs.